

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a0158bc680310a701.jsonl`  
- SHA-256: `58a3e56a7ea0264131a9085b22030ed5ff882d96f506a31ac7665cd23f0354fe`  
- Source modified: `2026-06-10T16:31:35+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:28:11.318Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Cr/Dr suffix sign-stripped in parse_amount ? an overdraft (Dr-balance) statement reconciles with every debit/credit INVERTED and persists as verified
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/validation.py (line 32, 45)
DESCRIPTION: parse_amount strips a trailing 'Cr'/'Dr' and returns the magnitude with no sign: '1,500.00 Dr' -> +1500.0. For an account whose balance is negative throughout (OD facility / EMI overdue ? Dr-suffixed balances are how Indian banks print these), the magnitude sequence of a negative-balance walk is exactly a sign-flipped positive walk, so the L2 mapping search (engine.py:285-293) finds that assigning the Withdrawals column to 'credit' and Deposits to 'debit' makes the arithmetic work, and map_table returns it verified=True. Empirically confirmed on an AXIS_PROFILE header-less matrix (exactly what cluster_to_matrix emits, since txn_line_predicate drops header lines): true events 'withdraw 500, deposit 300, withdraw 800' on balances -1000->-1500->-1200->-2000 printed as '1,500.00 Dr' etc. came back `verified: True, layer: L2, map {'debit': 3, 'credit': 2}` ? every direction inverted, balances stored positive. The total-reconcile bookend does NOT save this even if finding #1 is fixed: parse_statement_summary's regex also captures magnitudes only, and magnitudes are self-consistent under inversion. This is the worst-case class: wrong rows that still reconcile.
EVIDENCE QUOTED: validation.py:32: `\_CR\_DR\_SUFFIX = re.compile(r"(?i)\s\*(cr|dr)\s\*\$")`; :45: `text = \_CR\_DR\_SUFFIX.sub("", str(value).strip())...` (sign discarded). engine.py:43: `\_MONEY\_RE = re.compile(r"(?i)^\s\*[d,]+\.\d{1,2}(?:\s\*(?:cr|dr))?\$")`. Empirical: map_table on Dr-suffixed balances returned `verified: True layer: L2 map: {'txn\_date': 0, 'narration': 1, 'debit': 3, 'credit': 2, 'balance': 4}` with 'UPI OUT' stored as credit=500.0.
SUGGESTED FIX: Make parse_amount sign-aware: a 'Dr' suffix on a balance yields a negative value (Cr positive), or return (magnitude, suffix) and have the engine reject candidate mappings that contradict the printed suffix. Additionally, have map_table detect AMBIGUITY: if more than one candidate mapping passes the oracle (the inverted and correct mappings both pass for sign-symmetric ledgers), refuse to auto-select and quarantine.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers,

balances, person names, client secrets, transaction amounts) in your output.

- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Structured output only.

2. user (2026-06-10T16:28:16.655Z)

```
1      """The verifier ? reconciliation as a layout-independent truth oracle.
2
3      Reconciliation proves a parse is complete and correct without inspecting any sensitive
value1:
4      if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5      parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6      money ? extraction proposes, reconciliation disposes (see
7      [docs/parser-architecture.md]([../.../docs/parser-architecture.md]) §2, §5).
8
9      This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11     * per-row walk ? reconcile_running_balance: balance[i] == balance[i-1] ? debit
+ credit.
12     * total reconcile ? reconcile_total: opening + ?credit ? ?debit == closing
(works even
13     with no per-row balance, e.g. Axis savings).
14     * bookend ? verify: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15     * selection ? select_reconciling: among candidate concept mappings, pick the one
arithmetic
16     endorses (the seam the L2 mapping search plugs into).
17     * confidence ? Confidence + BookendReport.confidence: a recorded verdict,
not an
18     assumption.
19
20     Reports carry counts / booleans / a verdict only ? never an amount.
21     """
22
23     from __future__ import annotations
24
25     import re
26     from collections.abc import Sequence
27     from dataclasses import dataclass
28     from enum import StrEnum
29
30     from .concepts import ConceptRow, StatementConcepts
31
32     _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35     def parse_amount(value: object) -> float | None:
36         """Parse a statement money cell to a float.
37
38         Returns None only for a genuinely absent cell (None / empty / "-" /
non-numeric); "0.00" parses to 0.0. This deliberately distinguishes
39         absent from zero ? unlike base.safe_amount, which maps "0.00" to
40         None and would corrupt a real zero balance and the reconciliation math.
41         """
42
43         if value is None:
44             return None
45         text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46         if text in ("", "-"):
47             return None
48         try:
```

```

49         return float(text)
50     except ValueError:
51         return None
52
53
54 @dataclass
55 class ReconcileReport:
56     """Result of a running-balance walk. Counts only, never an amount."""
57
58     n: int # rows considered
59     compared: int # consecutive (balance-bearing) pairs actually checked
60     breaks: int
61     first_break: int | None = None
62
63     @property
64     def ok(self) -> bool:
65         return self.compared > 0 and self.breaks == 0
66
67
68 def reconcile_running_balance(
69     entries: Sequence[tuple[float | None, float | None, float | None]],
70     tol: float = 0.01,
71 ) -> ReconcileReport:
72     """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
73
74     ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
75     ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
76     is skipped (can't anchor on it). Because the walk spans the whole statement, a
77     clean result also rules out any row missed *between* two balances ? which is the
78     failure mode silent ``extract_tables()`` row-drops would otherwise hide.
79     """
80     breaks = 0
81     first_break: int | None = None
82     compared = 0
83     prev: float | None = None
84     for i, (debit, credit, balance) in enumerate(entries):
85         if balance is None:
86             continue
87         if prev is not None:
88             expected = prev - (debit or 0.0) + (credit or 0.0)
89             compared += 1
90             if abs(expected - balance) > tol:
91                 breaks += 1
92                 if first_break is None:
93                     first_break = i
94         prev = balance
95     return ReconcileReport(
96         n=len(entries), compared=compared, breaks=breaks, first_break=first_break
97     )
98
99
100 def _row_entries(
101     rows: Sequence[ConceptRow],
102 ) -> list[tuple[float | None, float | None, float | None]]:
103     """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
104     consumes."""
105     return [(r.debit, r.credit, r.balance) for r in rows]
106
107
108 def reconcile_total(
109     rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
110 ) -> bool:
111     """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
112
113     This is the oracle for statements with **no reliable per-row balance** (e.g. Axis

```

```

savings):
113         it checks the whole statement against its own opening/closing bookends without
needing each
114         row's running balance. ``opening`` and ``closing`` come from the bank's
SUMMARY/header
115         (``StatementConcepts``); the caller must have them.
116         """
117         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
118         return abs((opening + delta) - closing) <= tol
119
120
121     class Confidence(StrEnum):
122         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).
123
124         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
125         a check failed (or none could run); the parse is suspect and is quarantined rather
than trusted.
126         """
127
128         RECONCILED = "reconciled"
129         UNRECONCILED = "unreconciled"
130
131
132     @dataclass
133     class BookendReport:
134         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
135
136         ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
137         requires the per-row balance walk *plus* every available check to pass.
138         """
139
140         n: int
141         reconciles: bool
142         debit_count_ok: bool | None = None
143         credit_count_ok: bool | None = None
144         opening_ok: bool | None = None
145         closing_ok: bool | None = None
146
147         @property
148         def ok(self) -> bool:
149             checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
150             return self.n > 0 and self.reconciles and all(c is True for c in checks)
151
152         @property
153         def confidence(self) -> Confidence:
154             return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
155
156
157     def verify(
158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.
161
162         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
163         extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
164         figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
165         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed

```

```

opening).
166         """
167         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
168         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
169         if concepts.debit_count is not None:
170             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
171         if concepts.credit_count is not None:
172             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
173         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
174             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
175         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
176             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
177             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
178         return BookendReport(
179             n=len(rows),
180             reconciles=reconciles,
181             debit_count_ok=debit_count_ok,
182             credit_count_ok=credit_count_ok,
183             opening_ok=opening_ok,
184             closing_ok=closing_ok,
185         )
186
187
188     def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
189         """Among candidate concept-row mappings, return the index of the first that
reconciles.
190
191         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215

```

3. user (2026-06-10T16:28:17.480Z)

src\muneeem\parsers__init__.py

```
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\engine.py
src\muneem\parsers\base.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py
```

4. user (2026-06-10T16:28:26.103Z)

```
1      """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3      This is the *generic-first* half of the concept-core architecture (see
4      [docs/parser-architecture.md](../docs/parser-architecture.md) §4, §6).
5      Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
6      or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7      via a cheap-first ladder, with a profile supplying priors:
8
9      * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
10     * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
11     * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
12     plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
13     reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
14     makes the math work* rather than by its label.
15
16     The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
17     names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
18     (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
19     the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20     value; provenance is the winning layer + column map.
21     """
22
23     from __future__ import annotations
24
25     import itertools
26     import re
27     from collections.abc import Callable, Sequence
28     from dataclasses import dataclass, field
29
30     from .concepts import ConceptRow
31     from .validation import parse_amount
32
33     Cell = object # a table cell: str | None (we coerce defensively)
34     Row = Sequence[Cell]
35
36     # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
37     _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
38         re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy
39         re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy
40     )
41     # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
```

```

42     # for an amount column during content classification.
43     _MONEY_RE = re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")
44     _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
45
46     # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
47     # lower-cased header labels matched exactly (not by substring) during L0.
48     _BANK_ALIASES: dict[str, tuple[str, ...]] = {
49         "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
50         "value_date": ("value date", "val date", "value dt"),
51         "narration": (
52             "narration",
53             "particulars",
54             "description",
55             "transaction details",
56             "details",
57             "remarks",
58             "transaction",
59             "transaction remarks",
60         ),
61         "reference": (
62             "ref",
63             "reference",
64             "ref no",
65             "reference no",
66             "cheque no",
67             "chq no",
68             "instrument no",
69         ),
70         "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
71         "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
72         "balance": (
73             "balance",
74             "closing balance",
75             "running balance",
76             "closing bal",
77             "available balance",
78         ),
79     }
80
81
82     def _text(cell: Cell) -> str:
83         return "" if cell is None else str(cell).strip()
84
85
86     def _norm(label: str) -> str:
87         return re.sub(r"\s+", " ", label.strip().lower())
88
89
90     def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
91         return any(p.match(text) for p in patterns)
92
93
94     def _looks_like_money(text: str) -> bool:
95         return bool(_MONEY_RE.match(text))
96
97
98     @dataclass
99     class FamilySpec:
100         """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
101
102         The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
103         future instrument family (with its own concepts + reconciliation oracle) reuses the

```

```

same engine.
104     ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105     """
106
107     name: str
108     concepts: tuple[str, ...]
109     date_concepts: tuple[str, ...] # in order: the first is the primary (row-gating)
date
110     amount_concepts: tuple[str, ...] # money columns; the non-balance ones are the
"flows"
111     balance_concept: str | None # the running-balance concept, if the family has one
112     text_concepts: tuple[str, ...] # free-text columns; the first gets the widest text
column
113     default_aliases: dict[str, tuple[str, ...]]
114     build: Callable[[list[dict[str, str]]], list]
115
116
117     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
118         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134
135     BANK_FAMILY = FamilySpec(
136         name="bank",
137         concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
138         date_concepts=("txn_date", "value_date"),
139         amount_concepts=("debit", "credit", "balance"),
140         balance_concept="balance",
141         text_concepts=("narration", "reference"),
142         default_aliases=_BANK_ALIASES,
143         build=_build_bank_rows,
144     )
145
146
147     @dataclass
148     class MappingResult:
149         """The outcome of mapping one table: typed rows + how we got there (provenance)."""
150
151         rows: list # typed rows from family.build (e.g. list[ConceptRow])
152         mapped: list[
153             dict[str, str]
154         ] # per-row {concept: raw cell}, incl. any profile-declared extra columns
155         column_map: dict[str, int] # the winning concept -> column-index assignment
156         layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
157         verified: bool # did the oracle endorse it? (False if no oracle was supplied)
158
159
160     @dataclass
161     class Profile:

```



```

162     """Declarative per-issuer knowledge ? *data, not code* (architecture $6).
163
164     Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
165     accepted date shape, a positional fallback when there's no header, table anchors,
account-number
166     patterns, quirks, and (placeholder, wiring parked) the password rule.
167     """
168
169     name: str
170     aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
171     date_patterns: tuple[re.Pattern, ...] = ()
172     column_order: tuple[str, ...] | None = None
173     table_anchors: tuple[str, ...] = ()
174     account_patterns: tuple[re.Pattern, ...] = ()
175     quirks: frozenset[str] = frozenset()
176     password_rule: str | None = None # data only; sourcing is parked (DESIGN $8.2)
177
178     def first_account(self, text: str) -> str | None:
179         for rx in self.account_patterns:
180             m = rx.search(text)
181             if m:
182                 return m.group(1).strip()
183         return None
184
185
186     def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
187         """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
188         index: dict[str, str] = {}
189         for concept, labels in family.default_aliases.items():
190             for label in labels:
191                 index[_norm(label)] = concept
192         for concept, labels in profile.aliases.items():
193             for label in labels:
194                 index[_norm(label)] = concept
195         return index
196
197
198     def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
199         return profile.date_patterns or _GENERIC_DATE_PATTERNS
200
201
202     def _detect_header(
203         rows: list[list[Cell]], alias_index: dict[str, str]
204     ) -> tuple[int | None, dict[str, int]]:
205         """Find the header row (most exact alias hits, >=2) and its concept->column map."""
206         best_idx: int | None = None
207         best_map: dict[str, int] = {}
208         for i, row in enumerate(rows):
209             cmap: dict[str, int] = {}
210             for col, cell in enumerate(row):
211                 concept = alias_index.get(_norm(_text(cell)))
212                 if concept and concept not in cmap:
213                     cmap[concept] = col
214             if len(cmap) > len(best_map):
215                 best_map, best_idx = cmap, i
216         if len(best_map) < 2:
217             return None, {}
218         return best_idx, best_map
219
220
221     def _is_balance_marker(row: Row) -> bool:
222         return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)
223

```

```

224
225     def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
226         if date_col is not None:
227             cell = row[date_col] if date_col < len(row) else None
228             return _matches_date(_text(cell), patterns)
229         return any(_matches_date(_text(c), patterns) for c in row)
230
231
232     def _classify_columns(
233         data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
234     ) -> dict[int, str]:
235         """Tag each column date / amount / text / empty by majority content."""
236         kinds: dict[int, str] = {}
237         for c in range(ncols):
238             cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
239             if not cells:
240                 kinds[c] = "empty"
241                 continue
242             thresh = max(1, int(len(cells) * 0.6))
243             if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
244                 kinds[c] = "date"
245             elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
246                 kinds[c] = "amount"
247             else:
248                 kinds[c] = "text"
249         return kinds
250
251
252     def _base_map(
253         col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
254     ) -> dict[str, int]:
255         """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.
256
257         Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
258         goes to the family's first text concept (if any). Amount columns are left to the
search.
259
260         """
261         base: dict[str, int] = {}
262         date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
263         text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
264         for concept, col in zip(family.date_concepts, date_cols, strict=False):
265             base[concept] = col
266         if text_cols and family.text_concepts:
267             base[family.text_concepts[0]] = max(
268                 text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
269             )
270         return base
271
272     def _search_candidates(
273         base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
274     ) -> list[tuple[str, dict[str, int]]]:
275         """Enumerate plausible amount-column ? concept assignments (the L2 search space).
276
277         Family-agnostic: the "flows" are the family's amount concepts minus its balance
concept (for
278         banks: debit, credit). We try each amount column as the balance and assign the rest
to the
279         flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
Oct).
280
281         """
282         amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]

```

```

282     flows = [c for c in family.amount_concepts if c != family.balance_concept]
283     bal = family.balance_concept
284     cand: list[dict[str, int]] = []
285     if bal and len(amount_cols) >= len(flows) + 1:
286         for bcol in amount_cols:
287             rest = [c for c in amount_cols if c != bcol]
288             for assign in itertools.permutations(rest, len(flows)):
289                 cand.append(**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
290     if flows and len(amount_cols) >= len(flows):
291         for assign in itertools.permutations(amount_cols, len(flows)):
292             cand.append(**base, **dict(zip(flows, assign, strict=True)))
293     return [("L2", c) for c in cand[:24]]
294
295
296     def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
297         out: list[dict[str, str]] = []
298         for row in data:
299             out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
300         return out
301
302
303     def map_table(
304         table: Sequence[Row] | None, # Sequence (covariant) so list[list[str | None]]
callers fit
305         profile: Profile,
306         family: FamilySpec = BANK_FAMILY,
307         *,
308         oracle: Callable[[list], bool] | None = None,
309     ) -> MappingResult | None:
310         """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.
311
312         Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
313         ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
314         structurally-complete candidate is returned unverified (the caller reconciles
separately).
315         Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
316         was supplied but nothing reconciled.
317         """
318         if not table:
319             return None
320         rows_in = [list(r) for r in table if r is not None]
321         if not rows_in:
322             return None
323         alias_index = _alias_index(profile, family)
324         patterns = _date_patterns(profile)
325
326         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
327         start = header_idx + 1 if header_idx is not None else 0
328         date_col_hint = col_map_hdr.get("txn_date")
329         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
330             date_col_hint = profile.column_order.index("txn_date")
331
332         data = [
333             r
334             for r in rows_in[start:]
335             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
336         ]
337         if not data:

```

```

338         return None
339     ncols = max(len(r) for r in data)
340     col_kinds = _classify_columns(data, ncols, patterns)
341     base = _base_map(col_kinds, data, family)
342     flows = [c for c in family.amount_concepts if c != family.balance_concept]
343
344     candidates: list[tuple[str, dict[str, int]]] = []
345     if flows and all(c in col_map_hdr for c in flows): # L0: header pinned the flow
concepts
346         candidates.append(("L0", {**base, **col_map_hdr}))
347     elif col_map_hdr: # header present but didn't pin the flows: keep its hints, search
the rest
348         candidates.append(("L0", {**base, **col_map_hdr}))
349     if profile.column_order: # positional prior (no/partial header)
350         candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351     candidates += _search_candidates(base, col_kinds, family)
352
353     primary_date = family.date_concepts[0] if family.date_concepts else None
354     best: MappingResult | None = None
355     for layer, cmap in candidates:
356         if primary_date and primary_date not in cmap:
357             continue
358         if not (set(family.amount_concepts) & cmap.keys()):
359             continue
360         mapped = _apply_map(data, cmap)
361         typed = family.build(mapped)
362         if oracle is None:
363             return MappingResult(typed, mapped, cmap, layer, verified=False)
364         if oracle(typed):
365             return MappingResult(typed, mapped, cmap, layer, verified=True)
366         if best is None:
367             best = MappingResult(typed, mapped, cmap, layer, verified=False)
368     return best
369
370
371 __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372

```

5. user (2026-06-10T16:28:36.235Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
Axis specifics
6     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
mapping.
7
8     Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
reconcile
9     (parse/persist split + count only). No statement content is logged here.
10    """
11
12    from __future__ import annotations
13
14    import re
15    import sqlite3
16    from dataclasses import dataclass
17    from pathlib import Path
18
19    import pdfplumber

```

```

20
21     from muneem import __version__
22     from muneem import db as dbmod
23     from muneem.errors import ParseError
24     from muneem.redaction import redact_filename
25
26     from . import register_parser
27     from .base import StatementParser
28     from .profiles import AXIS_PROFILE
29     from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30     from .validation import Confidence, parse_amount
31
32     _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
33     _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
34
35
36     def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(AXIS_PROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, AXIS_PROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66         def parse_and_insert(
67             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68         ) -> int:
69             """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71             Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72             parser can't yet read surfaces loudly instead of saving mislabeled rows.
73             """
74             try:
75                 parsed = parse_axis_savings(pdf_path, password)
76             except ParseError:

```

```

77         raise
78     except Exception as exc:
79         # Filename only ? never statement text; original cause kept via `from`.
80         name = redact_filename(pdf_path)
81         raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83     if not parsed.verified:
84         dbmod.record_parse(
85             conn,
86             doc_id,
87             confidence=Confidence.UNRECONCILED.value,
88             parser="AxisParser",
89             parser_version=__version__,
90         )
91     return 0
92
93     for r in parsed.rows:
94         dbmod.insert_axis_transaction(
95             conn,
96             document_id=doc_id,
97             account_number=parsed.account_number,
98             txn_date=r.txn_date,
99             value_date=r.value_date or "",
100             particulars=r.narration,
101             debit=r.debit,
102             credit=r.credit,
103             balance=r.balance,
104         )
105         dbmod.record_parse(
106             conn,
107             doc_id,
108             confidence=Confidence.RECONCILED.value,
109             parser="AxisParser",
110             parser_version=__version__,
111         )
112     return len(parsed.rows)
113
114
115     # --- Credit card (no running balance; count only)
116     -----
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
130         for table in tables or []:
131             for row in table or []:
132                 if not row or len(row) < 9:
133                     continue
134                 date_str = str(row[0]).strip() if row[0] else ""
135                 if not _CC_DATE_RE.match(date_str):
136                     continue
137                 amount_str = str(row[8]).strip() if row[8] else ""
138                 is_cr = "Cr" in amount_str
139                 amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                 rows.append(

```

```

141         AxisCCTxn(
142             txn_date=date_str,
143             particulars=str(row[2]).strip() if row[2] else "",
144             merchant_category=str(row[7]).strip() if row[7] else "",
145             amount=amount,
146             type="Cr" if is_cr else "Dr",
147         )
148     )
149     return rows
150
151
152     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153         """Open an Axis credit-card statement and return (card_number, transactions)."""
154         card_number = "UNKNOWN"
155         rows: list[AxisCCTxn] = []
156         with pdfplumber.open(pdf_path, password=password) as pdf:
157             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159             for page in pdf.pages:
160                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
161             return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192             # record provenance (with no confidence verdict).
193             dbmod.record_provenance(
194                 conn,
195                 document_id=doc_id,
196                 source_type="pdf_statement",
197                 parser="AxisCreditCardParser",
198                 parser_version=__version__,
199                 mapping_layer="deterministic",
200             )

```

```
201         return len(rows)
202
```

6. user (2026-06-10T16:28:36.573Z)

```
1      """Coordinate clustering ? turn positioned words into a clean cell-matrix.
2
3      When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned
statement (the
4      Axis case: a date merged into the narration cell, column counts drifting between
vintages), we read
5      **word positions** directly. Words are grouped into visual rows by their ``top``; column
boundaries
6      are inferred from the vertical whitespace **shared across the transaction rows**; and
each word is
7      dropped into the column its x-centre falls in. Because columns come from geometry, a
leading date
8      sits in its *own* column instead of being glued to the narration.
9
10     Two robustness lessons baked in:
11
12     * **Infer columns from transaction-like rows only.** Headers, address blocks, and
summaries span the
13     page and would erase the inter-column whitespace. A caller-supplied ``keep_line``
predicate (e.g.
14     "has a date *and* a money token") restricts both column inference and the emitted
matrix to the
15     transaction band.
16     * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
empty
17     in *most* rows; a single long narration can't collapse two columns.
18
19     Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's
oracle still
20     decides whether the result is trustworthy. Pure geometry in, strings out ? no value is
logged.
21     """
22
23     from __future__ import annotations
24
25     from collections.abc import Callable, Sequence
26
27     Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
28
29
30     def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
31         """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
32         lines: list[list[Word]] = []
33         anchor_top: float | None = None
34         for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
35             if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
36                 lines[-1].append(w)
37             else:
38                 lines.append([w])
39                 anchor_top = w["top"]
40         return [sorted(line, key=lambda w: w["x0"]) for line in lines]
41
42
43     def infer_columns(
44         lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
45     ) -> list[tuple[float, float]]:
46         """Infer column x-extents from a coverage histogram over ``lines``.
47
48         An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
```



```

columns are
49     the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
50     to a few full-width lines: it thresholds on the *fraction* of rows, not on any
single one.
51     """
52     words = [w for line in lines for w in line]
53     if not words:
54         return []
55     x0 = min(float(w["x0"]) for w in words)
56     x1 = max(float(w["x1"]) for w in words)
57     nbins = max(1, int((x1 - x0) / bin_w) + 1)
58     cover = [0] * nbins
59     for line in lines:
60         marked: set[int] = set()
61         for w in line:
62             a = int((float(w["x0"]) - x0) / bin_w)
63             b = int((float(w["x1"]) - x0) / bin_w)
64             marked.update(range(max(0, a), min(nbins - 1, b) + 1))
65         for k in marked:
66             cover[k] += 1
67     thresh = max(1.0, min_frac * len(lines))
68     filled = [c >= thresh for c in cover]
69     cols: list[tuple[float, float]] = []
70     start: int | None = None
71     for i, f in enumerate(filled):
72         if f and start is None:
73             start = i
74         elif not f and start is not None:
75             cols.append((x0 + start * bin_w, x0 + i * bin_w))
76             start = None
77     if start is not None:
78         cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
79     return cols
80
81
82     def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
83         for i, (a, b) in enumerate(columns):
84             if a <= center <= b:
85                 return i
86         return min(
87             range(len(columns)),
88             key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
89         )
90
91
92     def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) ->
list[str]:
93         """Place each word into its column; join words that share a column (in x order)."""
94         cells = [" " for _ in columns]
95         for w in line:
96             idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
97             cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
98         return cells
99
100
101     def cluster_to_matrix(
102         words_by_page: Sequence[Sequence[Word]],
103         *,
104         line_tol: float = 2.5,
105         min_frac: float = 0.3,
106         keep_line: Callable[[list[str]], bool] | None = None,
107     ) -> list[list[str]]:
108         """Cluster words across pages into one cell-matrix with consistent columns.
109

```

```

110     ``keep_line`` (texts of a line ? bool) restricts both column inference and the
output to the
111     transaction band ? without it, full-width headers/footers swamp the column
structure. Columns
112     are inferred once across all kept lines (a statement keeps its layout); rows are
emitted in
113     page-then-top order.
114     """
115     lines: list[list[Word]] = []
116     for page in words_by_page:
117         lines.extend(group_lines(page, line_tol))
118     if keep_line is not None:
119         lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
120     if not lines:
121         return []
122     columns = infer_columns(lines, min_frac=min_frac)
123     if not columns:
124         return []
125     return [line_to_cells(line, columns) for line in lines]
126
127
128     __all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]
129

```

7. user (2026-06-10T16:28:45.309Z)

```

1     """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3     Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4     and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6     1. Extraction proposes candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7     coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8     2. The generic engine maps each candidate's columns onto the canonical concepts.
9     3. Reconciliation disposes: a parse is trusted only if its per-row balance walks, or
? when a
10     vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12     A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13     on its own, keep the rows the oracle endorses, and re-check the combined set (which
catches a
14     cross-table balance break, i.e. a missing transaction). The result is oracle-gated:
a layout we
15     can't yet read returns ``verified=False`` and the caller persists nothing ? never
misabeled rows.
16
17     No statement content is logged; only counts/booleans travel out.
18     """
19
20     from __future__ import annotations
21
22     import re
23     from dataclasses import dataclass, field
24     from pathlib import Path
25
26     import pdfplumber
27
28     from .clustering import cluster_to_matrix
29     from .concepts import ConceptRow, StatementConcepts
30     from .engine import Profile, map_table

```

```

31     from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33     _MONEY_TOKEN = re.compile(r"^~?[\d,]+\.\d{2}(:cr|dr)?$", re.IGNORECASE)
34     _OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
35     _CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
36
37
38     @dataclass
39     class SavingsParse:
40         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42         account_number: str
43         rows: list[ConceptRow] = field(default_factory=list)
44         verified: bool = False
45
46
47     def parse_statement_summary(text: str) -> StatementConcepts:
48         """Read opening/closing balance from statement text (for the total-reconcile
oracle).
49
50         Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
51
52         """
53         opening = _OPENING_RE.search(text)
54         closing = _CLOSING_RE.search(text)
55         return StatementConcepts(
56             opening_balance=parse_amount(opening.group(1)) if opening else None,
57             closing_balance=parse_amount(closing.group(1)) if closing else None,
58         )
59
60     def txn_line_predicate(profile: Profile):
61         """A line is transaction-like if it carries one of the profile's dates *and* a money
token.
62
63         Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
64         address blocks that would otherwise wreck column inference).
65
66         """
67         patterns = profile.date_patterns
68
69         def keep(texts: list[str]) -> bool:
70             has_date = any(p.match(t) for p in patterns) for t in texts)
71             return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
72
73         return keep
74
75     def reconcile_oracle(opening: float | None, closing: float | None):
76         """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""
77
78         def check(rows: list[ConceptRow]) -> bool:
79             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
80                 return True
81             if rows and opening is not None and closing is not None:
82                 return reconcile_total(rows, opening, closing)
83             return False
84
85         return check
86
87
88     def _reconstruct_by_balance_delta(
89         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01

```

```

90     ) -> list[ConceptRow]:
91         """Recover debit/credit from the running balance when columns won't split cleanly.
92
93         Needs only three things per row ? a date, the transaction amount, and the running
94         balance ? so
95         it tolerates messy column inference (a sliver column, an interleaved value-date
96         column, or a
97         single merged withdrawal/deposit column). Direction is the sign of the balance
98         change; the
99         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
100        every row, so
101        a misread or a dropped row still fails (it is *not* circular). Needs the printed
102        opening balance
103        to fix the first row's direction; returns ``[]`` if it can't form a clean series.
104        """
105        patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
106        rows = [(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
107        if not rows or opening is None:
108            return []
109        ncols = max(len(r) for r in rows)
110
111        def is_date(s: str) -> bool:
112            return any(p.match(s) for p in patterns)
113
114        def money(s: str) -> float | None:
115            return parse_amount(s) if _MONEY_TOKEN.match(s) else None
116
117        # date column = the leftmost of the (first few) columns with the most date cells
118        date_counts: dict[int, int] = {}
119        for r in rows:
120            for c in range(min(ncols, 3)):
121                if c < len(r) and is_date(r[c]):
122                    date_counts[c] = date_counts.get(c, 0) + 1
123        if not date_counts:
124            return []
125        top = max(date_counts.values())
126        date_col = min(c for c, n in date_counts.items() if n == top)
127
128        txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
129        if len(txn_rows) < 2:
130            return []
131
132        # A money column's money cells must outnumber its date cells ? so a sparsely-filled
133        deposit
134        # column still counts, but an interleaved value-date column (mostly dates) does not.
135        The
136        # running balance is the rightmost money column; the rest are the flow
137        (withdrawal/deposit).
138        money_cols = []
139        for c in range(ncols):
140            m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
141            d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
142            if m >= 1 and m > d:
143                money_cols.append(c)
144        if len(money_cols) < 2:
145            return []
146        balance_col = max(money_cols)
147        flow_cols = [c for c in money_cols if c != balance_col]
148
149        text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
150        narr_col = (
151            max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
152            if text_cols
153            else None
154        )

```

```

147
148     out: list[ConceptRow] = []
149     prev = opening
150     for r in txn_rows:
151         bal = money(r[balance_col]) if balance_col < len(r) else None
152         if bal is None:
153             return [] # a running balance on every row is required for the delta
154         amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
155         debit = credit = None
156         if amt is not None:
157             if bal < prev - tol:
158                 debit = amt
159             elif bal > prev + tol:
160                 credit = amt
161             else:
162                 debit = amt # zero net with an amount present ? reconciliation will
flag it
163         out.append(
164             ConceptRow(
165                 txn_date=r[date_col],
166                 narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
167                 debit=debit,
168                 credit=credit,
169                 balance=bal,
170             )
171         )
172         prev = bal
173     return out
174
175
176     def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
177         """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.
178
179         Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
180         and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
181         reconciles, so the caller can refuse to persist.
182         """
183         with pdfplumber.open(pdf_path, password=password) as pdf:
184             words_by_page = [
185                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
186             ]
187             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
188             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
189
190             account_number = profile.first_account(head) or "UNKNOWN"
191             summary = parse_statement_summary(head)
192             oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)
193
194             confirmed: list[ConceptRow] = []
195             for table in tables:
196                 res = map_table(table, profile, oracle=oracle)
197                 if res and res.verified:
198                     confirmed.extend(res.rows)
199             if confirmed and oracle(confirmed):
200                 return SavingsParse(account_number, confirmed, True)
201
202             matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))

```

```

203     clustered = map_table(matrix, profile, oracle=oracle)
204     if clustered and clustered.verified:
205         return SavingsParse(account_number, clustered.rows, True)
206
207     # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
208     #     debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
209     #     direction from the running balance and let the oracle cross-check amount ==
|?balance|.
210     delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
211     if delta and oracle(delta):
212         return SavingsParse(account_number, delta, True)
213
214     fallback = confirmed or (clustered.rows if clustered else [])
215     return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225

```

8. user (2026-06-10T16:28:45.525Z)

```

1     """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
$6).
2
3     A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4     concept?label aliases, the accepted date shape, a positional fallback when there's no
header,
5     table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
an issuer is
6     ideally *adding a profile*, not writing a parser.
7
8     `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
coordinate-
9     clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
table (see the
10     `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
engine-driven
11     parsers (coordinate clustering ? generic mapping ? total reconcile).
12     """
13
14     from __future__ import annotations
15
16     import re
17
18     from .engine import Profile
19
20     AXIS_PROFILE = Profile(
21         name="axis",
22         aliases={
23             # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
| Balance |
24             # Other Information. "Transaction" is the narration column (not the date).
25             "narration": ("transaction", "particulars"),
26             "debit": ("withdrawals", "withdrawal", "debit"),
27             "credit": ("deposits", "deposit", "credit"),
28             "balance": ("balance",),
29         },

```

```

30         date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),), # dd-mm-yyyy or
dd/mm/yyyy
31         # Layout / column-count varies between vintages, so there is NO reliable
column_order ? the
32         # generic content search + total reconcile do the work. Broadened account pattern:
the old
33         # `Account No:` regex missed real statements.
34         account_patterns=(
35             re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?s*(\d{6,})", re.IGNORECASE),
36             re.compile(r"A/c\s*No\.\.?s*[:\s]*(\d{6,})", re.IGNORECASE),
37         ),
38         quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39     )
40
41     AUBANK_PROFILE = Profile(
42         name="aubank",
43         aliases={
44             "narration": (
45                 "transaction",
46                 "particulars",
47                 "description",
48                 "narration",
49                 "transaction details",
50             ),
51             "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52             "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53             "balance": ("balance", "closing balance", "running balance"),
54         },
55         # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56         date_patterns=(
57             re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58             re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59         ),
60         account_patterns=(
61             re.compile(r"Account\s*(?:No|Number)\s*[:\.-]?s*(\d{6,})", re.IGNORECASE),
62             re.compile(r"A/c\s*No\.\.?s*[:\s]*(\d{6,})", re.IGNORECASE),
63         ),
64         quirks=frozenset({"engine-driven"}),
65     )
66
67     HDFC_PROFILE = Profile(
68         name="hdfc",
69         aliases={
70             "debit": ("withdrawal", "withdrawals"),
71             "credit": ("deposit", "deposits"),
72             "balance": ("closing balance",),
73         },
74         date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
75         table_anchors=("Narration", "Closing Balance"),
76         account_patterns=(re.compile(r"Account\s*Number\s*[:.]?s*(\d{6,})",
re.IGNORECASE),),
77         # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78         # by word-position clustering rather than through the generic cell-matrix engine.
79         quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80     )
81
82     __all__ = ["AXIS_PROFILE", "AUBANK_PROFILE", "HDFC_PROFILE"]
83

```

9. user (2026-06-10T16:29:31.161Z)

```

tests\parsers\test_axis.py:121:          ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food",
"250.00 Dr"],

```

```

tests\parsers\test_axis.py:122:          ["06/02/2026", "x", "Refund", "", "", "", "", "Food",
"100.00 Cr"],
tests\parsers\test_axis.py:128:    assert rows[0].amount == 250.0 and rows[0].type == "Dr" and
rows[0].merchant_category == "Food"
tests\parsers\test_axis.py:129:    assert rows[1].amount == 100.0 and rows[1].type == "Cr"
tests\parsers\test_hdfc.py:221:    HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and
reconcile fully.
tests\parsers\test_hdfc.py:236:    f"dr={report.debit_count_ok} cr={report.credit_count_ok}
"

```

10. user (2026-06-10T16:29:32.700Z)

```

src\muneem\parsers\hdfc.py:45:    BookendReport,
src\muneem\parsers\hdfc.py:109:ValidationReport = BookendReport
src\muneem\parsers\hdfc.py:364:    return verify(rows, summary, tol)
src\muneem\parsers\validation.py:15:* **selection** ? ``select_reconciling``: among candidate
concept mappings, pick the one arithmetic
src\muneem\parsers\validation.py:17:* **confidence** ? ``Confidence`` +
``BookendReport.confidence``: a recorded verdict, not an
src\muneem\parsers\validation.py:133:class BookendReport:
src\muneem\parsers\validation.py:157:def verify(
src\muneem\parsers\validation.py:159:) -> BookendReport:
src\muneem\parsers\validation.py:178:    return BookendReport(
src\muneem\parsers\validation.py:188:def select_reconciling(candidates:
Sequence[Sequence[ConceptRow]], tol: float = 0.01) -> int | None:
src\muneem\parsers\validation.py:211:    "BookendReport",
src\muneem\parsers\validation.py:213:    "select_reconciling",

```

11. user (2026-06-10T16:29:46.662Z)

```

.claude
.githubhooks
.github
.mypy_cache
.pytest_cache
.ruff_cache
.venv
docs
src
testdata
tests
tools
.coverage
.gitattributes
.gitignore
CLAUDE.md
DESIGN.md
Makefile
pyproject.toml
README.md
venv exists

```

12. assistant (2026-06-10T16:30:04.826Z)

```

"""Adversarial repro: Dr-suffixed (negative) balance walk vs map_table. Synthetic data only."""

```

```

from muneem.parsers.engine import map_table
from muneem.parsers.profiles import AXIS_PROFILE
from muneem.parsers.savings import parse_statement_summary, reconcile_oracle
from muneem.parsers.validation import parse_amount

```

1. parse_amount sign behaviour

```

print("parse_amount('1,500.00 Dr') =", parse_amount("1,500.00 Dr"))
print("parse_amount('1,500.00 Cr') =", parse_amount("1,500.00 Cr"))

```


2. Header-less matrix (what cluster_to_matrix emits): true events on an OD account open -1000; withdraw 500 -> -1500; deposit 300 -> -1200; withdraw 800 -> -2000.

Printed Indian-style: magnitude + ' Dr'. Columns: date | narration | deposits | withdrawals

```
matrix = [
    ["01-02-2026", "UPI OUT", "", "500.00", "1,500.00 Dr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "1,200.00 Dr"],
    ["03-02-2026", "UPI OUT", "", "800.00", "2,000.00 Dr"],
]
summary_text = "Opening Balance 1,000.00 Dr\nClosing Balance 2,000.00 Dr"
summary = parse_statement_summary(summary_text)
print("summary opening/closing =", summary.opening_balance, summary.closing_balance)
oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

res = map_table(matrix, AXIS_PROFILE, oracle=oracle)
print("verified:", res.verified, "layer:", res.layer, "map:", res.column_map)
for r in res.rows:
    print(" row:", r.txn_date, repr(r.narration), "debit=", r.debit, "credit=", r.credit,
"balance=", r.balance)
```

3. Same but WITH a header row (extract_tables path) ? does L0 failing fall through to i

```
table_with_header = [
    ["Tran Date", "Transaction", "Deposits", "Withdrawals", "Balance"],
    ["01-02-2026", "UPI OUT", "", "500.00", "1,500.00 Dr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "1,200.00 Dr"],
    ["03-02-2026", "UPI OUT", "", "800.00", "2,000.00 Dr"],
]
res2 = map_table(table_with_header, AXIS_PROFILE, oracle=oracle)
print("with header -> verified:", res2.verified, "layer:", res2.layer, "map:", res2.column_map)
```

4. Control: positive (Cr-suffixed) walk must map correctly.

```
matrix_cr = [
    ["01-02-2026", "UPI OUT", "", "500.00", "500.00 Cr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "800.00 Cr"],
]
oracle_cr = reconcile_oracle(1000.0, 800.0)
res3 = map_table(matrix_cr, AXIS_PROFILE, oracle=oracle_cr)
print("Cr control -> verified:", res3.verified, "layer:", res3.layer, "map:", res3.column_map)
```

5. Mixed-sign walk (crosses zero): should fail both mappings -> quarantined (fail-safe?)

```
matrix_mixed = [
    ["01-02-2026", "UPI OUT", "", "500.00", "500.00 Cr"],
    ["02-02-2026", "UPI OUT", "", "800.00", "300.00 Dr"],
]
oracle_mx = reconcile_oracle(1000.0, -300.0)
res4 = map_table(matrix_mixed, AXIS_PROFILE, oracle=oracle_mx)
print("mixed-sign -> verified:", res4.verified if res4 else None, "layer:", res4.layer if res4
else None)
```

13. user (2026-06-10T16:30:04.837Z)

File created successfully at: C:\Users\avidu\Projects\muneem\tools_verify_drcr_repro.py (file state is current in your context ? no need to Read it back)

14. user (2026-06-10T16:30:13.656Z)

```
parse_amount('1,500.00 Dr') = 1500.0
parse_amount('1,500.00 Cr') = 1500.0
summary opening/closing = 1000.0 2000.0
verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 2, 'credit': 3,
'balance': 4}
row: 01-02-2026 'UPI OUT' debit= None credit= 500.0 balance= 1500.0
```

```
row: 02-02-2026 'NEFT IN' debit= 300.0 credit= None balance= 1200.0
row: 03-02-2026 'UPI OUT' debit= None credit= 800.0 balance= 2000.0
with header -> verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 2,
'credit': 3, 'balance': 4}
Cr control -> verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 3,
'credit': 2, 'balance': 4}
mixed-sign -> verified: False layer: L2
```

15. assistant (2026-06-10T16:30:50.387Z)

The repro confirms the finding exactly ? and slightly worse than claimed. Let me clean up the temp script.

16. user (2026-06-10T16:30:53.123Z)

```
M docs/feedbackJune2026.md
?? docs/feedback-antigravity-response.md
```

17. user (2026-06-10T16:31:27.491Z)

Structured output provided successfully

18. assistant (2026-06-10T16:31:35.424Z)

Verification complete.